Ray-dispatch launch-model measurements on Arc Pro B70 (Xe2): findings and questions

Version 1.7 — 2026-08-05 (changelog at the end)

We develop an open-source hybrid renderer (frustracer, https://github.com/fechols/frustracer) that runs the *same* ray-traced frame through several dispatch models — a compute kernel with inline `RayQuery`, and a DXR `DispatchRays` pipeline at several inlining levels — from one shader source, with correctness gates locking the arms to one image (exact-zero counters where the hardware permits bit identity; tight statistical bounds across the watertight-vs-Moller-Trumbore intersector difference), so a fast arm cannot be fast by computing less. On the B70 that instrument produced: one result we could not explain from public documentation (finding 1), magnitudes for two effects your RT developer guide predicts qualitatively (findings 2-3), a register-pressure decomposition measured from inside the shaders (finding 4), a same-code launch-regime pricing curve that we believe closes finding 1's mechanism (finding 5), a measured answer to the many-record-SBT counterfactual (the Q4 section), and one crash that looks like a driver bug.

Environment of record

GPU	Intel Arc Pro B70 (Battlemage / Xe2); caps: RT tier 1.1, SM 6.8, wave lanes 16..32, WorkGraphsTier 1.0
Drivers	32.0.101.**8805** (all August rows); July rows on 32.0.101.8515 are marked
OS	Windows 11 Pro 10.0.26200
Control GPU	RTX 4090 in the same box (adapter selection explicit, verified by name per run)
Timing	D3D12 timestamp queries around per-pass markers, 120-frame windows, warm shader cache (the async recompile is characterized and waited out), deterministic camera path, 2+ interleaved reps (B70 repeatability ±0.002 ms)

Method: what is held constant

All arms share one HLSL source tree (shading and intersection pasted into each compile unit), one acceleration structure, one camera path, one sample sequence:

- **compute** — `cs_6_5` kernel, primary + all secondary rays as inline `RayQuery`;
- **mode 2** — `lib_6_5` raygen, `DispatchRays` as a bare launch grid, *zero* `TraceRay`;
- **mode 1** — primary `TraceRay` -> closest-hit, secondaries inline in the hit shader;
- **mode 0** — the by-the-book all-`TraceRay` pipeline (identifier-only records; a single-record SBT, which matters in finding 2).

Finding 1 — DispatchRays costs 2-3x on the same code with zero TraceRay (audited + re-measured 2026-08-05)

Large scene (34.4M triangles; alpha-cutout + tinted-transmission candidate loops, 100-byte material records, large bindless texture tables), identical full-screen traversal, driver 8805, same parked pose. Originally recorded 1.28 ms compute / 2.59 mode 2 / 3.13 mode 1 (2026-08-01); because this is the brief's spearhead claim we then re-measured

and adversarially audited it end-to-end on the same driver. It SURVIVES, with sharper numbers and three corrections of our own accounting:

- **"Zero TraceRay" is artifact-proven, not asserted.** The mode-2 library disassembles to ZERO `dx.op.traceRay` DXIL operations — 16 live `rayQuery_TraceRayInline` as the positive control — on both the slim and the fat-scene configurations, with a source-level guard pin in CI.
- **The recorded comparison was bracket-asymmetric IN THE CLAIM'S FAVOR**: the compute row's timing bracket included root binds and two cache-fill dispatches that the `DispatchRays` row's bracket excluded (~3%). Bare-dispatch-vs-bare-dispatch, the large scene re-measures:

arm	ms (2 reps)	ratio
compute kernel (bare dispatch)	**1.178 / 1.180**	1.0x
mode 2 `DispatchRays` (zero TraceRay)	**2.808 / 2.815**	**2.38x**
mode 1	3.106 / 3.110	2.64x

- **The build lottery is ONE-SIDED, and the band never crosses below 2x.** Across three comment-level rebuilds of identical source, the compute kernel repeats to ±0.001 ms while the mode-2 raygen swings up to ±29% (procedural 1.18-1.66 ms; San Miguel 1.81-2.22). The like-for-like ratio bands **2.07-2.91x (procedural), 2.68-3.30x (San Miguel), 2.38x on the large scene at the current draw** — "~2x" is the FLOOR of the band. The codegen instability finding 4 predicted lives entirely in the DXR door; the compute door never moves.
- **The dead-exports control.** At mode 2 the fat closest-hit/miss/any-hit entries were still compiled and EXPORTED (dead code — no ray can reach them), an uncontrolled confound in every earlier capture. A raygen-ONLY state object (same DXIL blob, recursion depth 0, null miss/hit SBT tables) still pays **1.93x on the large scene** — the tax is the raygen's own hosting, confirming finding 5's attribution — but removing the dead exports recovers a dose-responsive **0% (slim scene) / 12% (San Miguel) / 18% (large scene, 0.53 ms)** of the ray dispatch on the B70 (4090: no effect either scene). The driver spends real per-dispatch time provisioning exports that cannot execute — possibly actionable on its own.
- One phrasing correction of ours: "register-fat shaders" is no longer the right qualifier. Per finding 5, the kernel's own ~50 live floats price from float zero, so SLIM scenes pay the tax too (2.1-2.9x across draws); fatness raises the absolute cost and adds the dead-exports increment.

That the tax scales with shader FOOTPRINT, not dispatch count, is pinned by a thin/fat A/B on the same pipeline (on master as `--dxr-inline 3`): a THIN raygen — bare-hit primary `TraceRay` writing a 20 B record, no shading — costs 0.23-0.35 ms full-screen on our small scenes and 0.54 ms on the large scene, through the same `DispatchRays` and acceleration structure that cost ~2.8 ms with the fat shader resident. Ablations agree: compiling out all secondary-ray TRAVERSAL from mode 1 (rng draws and control flow kept) collapses it 2.40 -> 0.48 ms — BELOW the full compute kernel — with sub-additive per-category savings (a register/occupancy signature, not ray work).

Ruled out by construction or measurement: ray dispatch itself, the scene/AS, shader source, fixed launch overhead (the thin raygen IS the launch + traversal floor; ExecuteIndirect overhead measured ~11 us across 8 back-to-back dispatches — your EI-in-hardware claim checks out), cold compiles, our own traversal structure, image divergence (the arms are gated to one image), stack reservation (finding 5), SIMD width at the exact configuration (the compute kernel AND the mode-2 raygen both report compiled SIMD16 on the fat scene), and now dead-export provisioning (measured and subtracted — the raygen-only control above). The remaining question is finding 5's: what the launch regime does to the register/scratch budget. `IGC_ShaderDumpEnable` (env and both registry locations, cache-proof fresh compile) remains inert.

Finding 2 — your guide predicts this one; here are the magnitudes

The Thread Sorting Unit sorts by shader record; our SBT had effectively one record, so the recursive pipeline paid full repacking for zero coherence benefit. Measured (deterministic 600-frame lap, 1080p, 1 spp, GPU frame span, July driver):

scene	mode 0	mode 1	mode 2
procedural 80k	9.05	2.35	1.41
stress (5000 objects)	5.30	1.64	1.22
San Miguel low-poly 5.6M	6.75	1.94	1.29

Secondary-`TraceRay` dispatch was 68-84% of the whole pipeline's cost — and the same proportion holds on the RTX 4090 at its own scale (1.34 -> 0.26 procedural), so this is not Arc-only; Arc's multiplier is ~1.4-1.5x NVIDIA's. (Mode-2 points carry finding 1's build-variance caveat.) The counterfactual this table begs — a genuinely many-record, material-sorted SBT — is now built and measured: see the Q4 section. SER remains out of reach at the reported SM 6.8.

Finding 3 — per-sample occupancy signature

At higher samples-per-pixel, mode 1's *marginal* cost per extra sample on the B70 is 2.2 ms/sample vs mode 2's 1.11 — the candidate-loop-fattened closest-hit shader pays occupancy per sample where the fused raygen pays once. Finding 4 pins the mechanism.

Finding 4 — the register cliff, measured from inside the shaders

With the ISA-dump routes inert, we instrumented the kernels themselves: each real kernel reports its COMPILED `WaveGetLaneCount()` (the per-shader SIMD width the compiler actually chose — a trivial probe kernel reads 32 at every group shape we dispatch), and a synthetic register ballast injects N provably-live floats into our cheapest kernel (a loop recurrence consuming the traced hit distance, folded under a branch on a constant-buffer value that is never true at runtime — not dead-codeable, image bit-identical).

The width table (8805; RTX 4090 control in parentheses):

kernel	shape	B70 width
fat RayQuery kernels (hybrid leaf, hemisphere, per-pixel reference, deferred shade)	cs_6_5	**16** (32)
DXR raygen — even the THIN bare-hit mode-3 arm	lib_6_5	**16** (32)
slim no-ray kernels (sky fill, tile recursion)	cs_6_5	32 (32)

Three consequences. (1) Every RayQuery-carrying kernel compiles SIMD16 on the B70 — half of our own question 2 answered, and hypothesis 1 partly confirmed: a raygen that only fires one bare-hit ray and stores 20 bytes still gets SIMD16, so the RT-stage narrowing looks regime-driven, not footprint-driven. (2) Our fat compute kernels are ALSO 16, so the reference-vs-deferred 1.9x gap inside compute (hypothesis 2's datum) is not width — which leaves spill. (3) The ballast sweep locates the spill knee: per-live-float cost is ~1.5-2 us up to +48 floats, breaks ~3x between **+56 and +60** (0.704 -> 0.785 ms in one 4-float step), and accelerates past it (+160 floats = 2.6x baseline). Our reference kernel sits ~56-60 live floats below the allocator's edge, and the deferred kernel behaves like the reference plus ~100 ballast floats. A feature-strip sweep on the deferred kernel shows the cost is a THRESHOLD, not a sum: stripping the reflection arm saves 0.49 ms, stripping the glass chain saves 0.49 ms *on a scene with zero transmissive geometry*, yet

stripping everything saves only 0.78 (singles sum 1.22) — shedding EITHER arm's live state clears the same spill edge. No strip flips 16 -> 32; SIMD16 appears sticky for RayQuery kernels on this driver. This threshold structure is, we believe, also the mechanism behind finding 1's build-to-build variance: a kernel sitting AT the edge tips either way on comment-level rebuilds.

Finding 5 — the launch regime prices live state from float zero: finding 1's mechanism, closed

The ballast of finding 4 is now injectable into EITHER host of the same code: `FR_BALLAST=N` targets the compute reference kernel, `FR_BALLAST=dxr:N` the zero-TraceRay mode-2 raygen — identical shader source, identical compiled SIMD16 (finding 4's width report confirms per run), same scene, same binary, same day. Two cost-vs-live-floats curves whose difference can only be the launch regime. Kernel time in ms (deterministic 600-frame lap, small procedural scene; the same maiden-discard/last-table discipline):

N ballast floats	B70 compute	B70 raygen	4090 compute	4090 raygen
0	0.610	1.646	0.270	0.260
16	0.638	2.097	0.270	0.427
32	0.660	2.799	0.263	0.594
64	0.780	4.400	0.433	0.952
96	0.990	6.040	0.906	1.385
160	2.078	10.675	~1.2	5.122

The compute curves have HEADROOM then a knee (B70: 1.7 us/float to +56, then the finding-4 break; 4090: free to +32). **The raygen curves have no knee at all — they price live state from the first float** (B70 ~20-45 us/float, accelerating; 4090 ~10 us/float, with a second cliff past +128). And the arithmetic closes finding 1: the B70's N=0 gap (1.646 − 0.610 ≈ 1.0 ms) is the kernel's own ~50 live floats at the regime's rate — the entire baseline DispatchRays tax is live state x regime pricing, no residual left for scheduling or launch. The shape is cross-vendor; the severity is Intel's: NVIDIA's raygen budget still covers this kernel's own state (baseline parity, 0.260 vs 0.270), the B70's is already exceeded by it. Width never flips in any cell — this is spill/scratch traffic at constant SIMD16, not width.

Two confirmations. **Real shader state behaves like ballast**: on San Miguel (the cutout+transmission-fat shaders), compiling out the secondary-ray machinery brings DispatchRays to compute parity (0.696 vs 0.710 ms; the fat baselines were 1.908 vs 0.710), and the SAME code removal is worth 34x more under DispatchRays than under compute (stripping the reflection arm: −0.023 vs −0.777 ms) — with finding 4's threshold non-additivity reproduced in the DXR column. **The pipeline stack is not the reservoir**: we also read `GetShaderStackSize` / `GetPipelineStackSize` per export (and can clamp via `SetPipelineStackSize`) — the B70's defaults are small and honest (64 B-2.2 KB; the spec formula is visible in the numbers), so the pricing is not a bloated stack reservation. One corroborating gem from that API: the all-TraceRay mode-0 closest-hit reports **1056 B = 264 floats of live state preserved around every secondary TraceRay** (4090: 544 B) — the driver's own accounting of why finding 2's mode-0 column is what it is.

What this leaves open is exactly one question: what does the ray-dispatch launch regime do to the per-thread register/scratch budget that compute hosting does not? A halved GRF allocation (128 vs 256 mode), ray-state co-residency in the GRF, or a different scratch-backing path would each produce this curve; we cannot distinguish them without compiler visibility.

Q4 follow-through: the many-record sorted-SBT ladder, measured

`--dxr-sbt 0..3` runs the counterfactual as a ladder, each rung one mechanism. Eight field-derived material classes partition the scene into per-class TLAS instances (`InstanceContributionToHitGroupIndex = class x 3` into a class-major SBT; every `TraceRay` call site untouched). Rung 1: 8 `ExportToRename` aliases of the ONE fat closest-hit — identical code, distinct identifiers, pure sort keys. Rung 2: one specialized DXIL library per class, provably-dead shading arms stripped — thin records, real register relief (the mechanism finding 4 measures). Rung 3: reflection/glass continuations become real `TraceRay`s landing in the hit surface's OWN class record (occlusion stays inline; declared recursion 5) — the shape the TSU is designed around.

Same protocol as finding 2 (2026-08-04, two reps forward-then-reversed, Arc after its 1600-frame warm-up; B70 8805, 4090 32.0.16.1062). Frame span, ms, at `--dxr-inline 0` — the TSU's regime — on default / stress / San Miguel:

rung	B70 spp=1	B70 marginal ms/sample	4090 spp=1
0 — one fat record	8.02 / 5.31 / 6.79	7.02 / 4.54 / 5.92	1.17 / 0.83 / 1.15
1 — alias (sort keys only)	8.60 / 5.39 / 6.67	7.61 / 4.64 / 5.82	1.18 / 0.84 / 1.16
2 — specialized	2.51 / 2.38 / 2.17	1.93 / 1.74 / 1.65	0.25 / 0.34 / 0.24
3 — recursive per-class	**1.49 / 1.78 / 1.29**	**0.93 / 1.03 / 0.81**	0.19 / 0.27 / 0.20

Four readings, offered as the Q4 answer. **Sort keys alone bought ~0 on both vendors** — sorting identical fat shaders has nothing to gain (at our scale: 8 classes, 80k-5.6M tris; content with dozens of genuinely divergent hit shaders may differ — exactly what we'd like to compare notes on). **Specialization is the prize**: thin per-class hit shaders recover 55-80% of the by-the-book pipeline's cost on BOTH vendors — most of the "launch-model tax" was the fat uber shader hosted in RT stages, and it largely vanishes when the hosted records are thin. (This is your own developer guide's page-22 bullet — "avoid spills at re-packing points by minimizing live values across trace calls" — run as an experiment with the magnitudes attached; our rung 3 also refines its tail-recursion rider: the recursive per-class dispatch keeps live state across every trace call and still reaches parity, because what matters is the SIZE of the live set, of which tail recursion is the zero case.) **Recursive per-class dispatch lands the textbook pipeline at parity with our best inline hybrids** (B70 1.49 vs same-day inline-3's 1.40; marginals improve 5-7x) — one flagged confound: rung 3 over rung 2 at inline 0 also moves occlusion inline, partly restating finding 2. **Specialization stabilizes Arc codegen**: every configuration whose repeats spread >15% is a fat-record one; specialized rows repeat tight — consistent with finding 4's cliff (thin shaders sit safely below it). Practical residue: at the shipping `--dxr-inline 1`, specializing the one record the primary dispatches is still worth −50 to −60% on the B70 (2.22/1.72/2.44 -> **1.05/0.87/0.97**) — the fastest DXR configuration we have measured on Arc, though our compute-hosted tracer still wins outright (0.64/0.78 recorded). Known rung-2 approximation: a record's flattened loop also shades continuation surfaces of other classes (worst measured error 9.6e-3 radiance at a glass-heavy pose; rung 3 is exact by construction). Rename semantics are vendor trivia worth recording: NVIDIA folds the 8 aliases to ONE shader identifier, the B70 mints all 8 distinct, and an AMD iGPU driver (32.0.21018.14) access-violates in `CreateStateObject` on any renamed export.

Bug report — DispatchGraph access violation (work graphs)

- Device reports WorkGraphsTier 1.0; the state object builds; backing ask 517.62 MB (min == max == 542769152, granularity 1).
- First `DispatchGraph` takes an access violation (0xC0000005), debug layer AND GPU-based validation silent. Reproduced identically on 8515 and 8805.
- The identical graph runs on the RTX 4090, bit-identical to our ExecuteIndirect ladder (same-seed image diff exactly 0.0), with an 82 MB backing ask — so we believe the graph is well-formed. Repro is one env var (`FR_WORKGRAPH=1`); happy to provide a standalone capture.

Corroborating observations

- Our ray-shooting kernels carry zero groupshared memory (per your LDS-shares-L1-with-RTU note); the LDS-carrying recursion kernels shoot no rays.
- Wave-intrinsic aggregation of queue atomics is a small cross-vendor REGRESSION here (~+3% frame span / ~+10% on the affected pass on the B70; we are reverting to plain atomics). A correction to our own method: an earlier "neutral" verdict came from a disable lever that reached only one of two aggregation sites.
- Inline `RayQuery` throughput on the B70 is competitive: our compute-hosted tracers beat every DXR arm on Arc at the shipping 1-spp config on every scene (re-verified parked AND under camera motion on the large scene: 3.3-3.5 ms vs mode-1 DXR's 5.4, same TLAS, same ~1.4 ms animation refit), while Ada prefers the DXR pipeline — the balance genuinely inverts by vendor. Consistent with findings 1-4, the renderer now defaults its DXR pipeline to zero-TraceRay mode 2 on Intel adapters (4.77 vs 5.36 ms large-scene span, plus finding 3's marginals) while NVIDIA keeps mode 1.

Hypotheses — explicitly guesses, ranked

1. The raygen runs under the bindless-thread-dispatch launch regime: narrower SIMD width and/or per-thread RT-stack residency and a different occupancy calculus. Finding 4 confirmed the width half (SIMD16 even when THIN); finding 5 now measures the residual — the regime prices live state from the first float, at 2-4x the compute spill rate, with no headroom — which is the signature of a smaller effective per-thread register/scratch budget (halved GRF mode? ray-state co-resident in the GRF?), and rules out scheduling and launch overhead as the residual.
2. Compiler path: `lib_6_5` vs `cs_6_5` through different IGC entrances. One datum complicates a pure two-doors story: our deferred-shading `cs_6_5` kernel (strictly less work than the compute arm) runs 1.9x slower (1.12 vs 0.60 ms) — finding 4 shows both are SIMD16 and puts the deferred kernel past the spill knee, so a knife-edge cliff exists on the compute door too; the RT-stage door just sits past it far more often.
3. Driver per-dispatch overhead — largely ruled out (the tax scales with per-pixel work, not dispatch count).

Questions

1. Finding 5 reduces finding 1 to one sharp question: same code, same SIMD16, the compute host gives ~56 live floats of headroom before its spill knee and the `DispatchRays` host gives ZERO — every float priced at 20-45 us from the start. What does the ray-dispatch launch regime do to the per-thread register/scratch budget (large-GRF mode denied? ray/RTU state co-resident in the GRF? a different scratch path?), and is there any driver or compiler lever that returns that headroom — or is it silicon? The two-command knee sweep in the repro section reproduces the curves.
2. We measured the SIMD-width half ourselves (finding 4: RayQuery kernels all SIMD16, thin raygen included; spill knee at baseline + ~56-60 live floats). Can you confirm our spill reading for the deferred-shade kernel, and is there ANY supported route to per-shader ISA/GRF/spill stats on shipping drivers? Both the `IGC_ShaderDumpEnable` env route and its registry spellings (global AND adapter class key) are inert on 8805 with a provably-fresh compile.
3. Is the DispatchGraph AV a known issue? We can hand you a minimal repro.
4. For workloads shaped like ours — thin SBT, fat uber-shader — would you expect a many-hit-group TSU-sorted pipeline to beat inline-in-compute? We ran the experiment (Q4 section): sort keys ~0, specialization recovered most of the tax, recursion reached parity with inline — does that decomposition match your model of the TSU's value?

5. Anything above you'd expect to change materially on a near-term driver? We stamp every number with a driver version and re-run on updates; happy to be a regression canary.

Repro

All CLI flags / env levers on the renderer (Windows, D3D12):

```
cargo run --release -- --spin path --gpu --spin-plain --gpu-timing --prefer-intel    # compute arm
cargo run --release -- --spin path --dxr --dxr-inline 2 --gpu-timing --prefer-intel  # mode 2
cargo run --release -- --spin path --dxr --dxr-inline 1 --gpu-timing --prefer-intel  # mode 1
cargo run --release -- --spin path --dxr --dxr-inline 0 --gpu-timing --prefer-intel  # mode 0

# The Q4 ladder (rung 3 requires --dxr-inline 0):
cargo run --release -- --spin path --dxr --dxr-inline 0 --dxr-sbt 1 --gpu-timing --prefer-intel  # sort
keys
cargo run --release -- --spin path --dxr --dxr-inline 0 --dxr-sbt 2 --gpu-timing --prefer-intel  #
specialized
cargo run --release -- --spin path --dxr --dxr-inline 0 --dxr-sbt 3 --gpu-timing --prefer-intel  #
recursive
```

(`--gpu` without `--spin-plain` runs our hybrid quadtree tracer, a different algorithm — the compute arm above is the plain per-pixel reference kernel.) Findings 4-5's instruments: `FR_WIDTH=1` (any run then prints a per-kernel `width (gpu): leaf=16 sky=32 ...` line); `FR_BALLAST=N` (N in 1..=256 live floats into the reference kernel, image bit-identical; sweep N against the `reference` timing row to reproduce the compute knee); `FR_BALLAST=dxr:N` (the same ballast in the mode-2 raygen — sweep against the `dxr-rays` row under `--dxr-inline 2` to reproduce finding 5's regime curve); and every DXR construction prints a `dxr stack:` line (per-export `GetShaderStackSize` + the default pipeline stack; `FR_DXR_STACK=min|<bytes>` overrides via `SetPipelineStackSize`). Finding 1's audit instruments: the `reference-kernel` timing row is the compute arm's bare dispatch (the like-for-like twin of `dxr-rays` — the outer `reference` row also contains binds and two cache fills); `FR_DXR_LEAN=1` builds the mode-2 raygen-only state object (recursion 0, null miss/hit tables — the dead-exports control); `FR_REF=1` starts an interactive session in the reference arm for large-scene captures (the scene our `--spin` harness cannot load). The work-graph AV: current builds refuse `FR_WORKGRAPH=1` on a picked Intel adapter precisely because of the crash — repro requires deleting that one-line refusal arm (grep `FR_WORKGRAPH` in `src/gpu/trace.rs`), or take our standalone capture. The benchmark is deterministic (camera pose a pure function of frame index), prints per-pass GPU-ms tables, and the correctness suites (`--check-gpu`, `--check-dxr`) gate the arms against each other bit-exactly where the hardware permits. We can provide exact poses, scripts, and raw logs.

Changelog

- **1.7** (2026-08-05): finding 1 adversarially audited and re-measured — the zero-TraceRay claim proven at the DXIL level; the timing brackets made like-for-like (the old compute bracket was ~3% inflated in the claim's favor); the build lottery banded across three rebuilds (2.07-3.30x, one-sided — the compute door repeats to ±0.001 ms) with "~2x" as the band FLOOR; the dead-exports control added (a raygen-only state object still pays 1.93x on the large scene, and removing dead exports recovers 12-18% of the ray dispatch on fat scenes — B70 only); the "register-fat" qualifier retired per finding 5; the guide's page-22 live-values bullet credited in the Q4 section.
- **1.6** (2026-08-04): finding 5 added — the knee-vs-knee launch-regime pricing curves (`FR_BALLAST=dxr:N`), the stack-size control, the strip-to-parity proof; finding 1's mechanism closed; hypothesis 1 and question 1

sharpened accordingly.

- **1.5** (2026-08-04): condensed edition; changelog moved here and shortened.
- **1.4** (2026-08-04): finding 4 added (per-kernel compiled SIMD widths, the spill knee, threshold cost structure); ShaderDump registry routes verified dead; question 2 sharpened.
- **1.3** (2026-08-04): the Q4 sorted-SBT ladder built and measured.
- **1.2** (2026-08-04): corrections pass — a stale small-scene control retired, build-variance caveats added, the wave-atomics verdict corrected, repro commands fixed.
- **1.1** (2026-08-04): driver-8805 re-captures; work-graph AV re-confirmed on 8805.
- **1.0** (2026-08): initial draft (July rows, driver 8515).